

Comandos Básicos de Git (Control de Versiones)

Comando	Descripción	Ejemplo de uso
<code>git init</code>	Inicializa un nuevo repositorio Git en la carpeta actual.	<code>git init</code>
<code>git clone <url></code>	Clona (descarga) un repositorio remoto a tu máquina local.	<code>git clone https://github.com/usuario/proyecto.git</code>
<code>git status</code>	Muestra el estado actual de los archivos (modificados, sin seguimiento, preparados).	<code>git status</code>
<code>git add <archivo></code>	Agrega un archivo al área de preparación (staging area).	<code>git add index.html</code>
<code>git add .</code>	Agrega todos los archivos modificados al área de preparación.	<code>git add .</code>
<code>git commit -m "mensaje"</code>	Guarda los cambios en el historial del repositorio con un mensaje descriptivo.	<code>git commit -m "Agrega nueva función de login"</code>
<code>git log</code>	Muestra el historial de confirmaciones (commits).	<code>git log</code>
<code>git diff</code>	Muestra los cambios realizados en los archivos antes de confirmar.	<code>git diff</code>
<code>git branch</code>	Muestra las ramas existentes o crea una nueva.	<code>git branch nueva-rama</code>
<code>git checkout <rama></code>	Cambia a otra rama existente.	<code>git checkout main</code>

Comando	Descripción	Ejemplo de uso
<code>git merge <rama></code>	Fusiona los cambios de una rama con la rama actual.	<code>git merge desarrollo</code>
<code>git remote add origin <url></code>	Vincula el repositorio local con un repositorio remoto.	<code>git remote add origin https://github.com/usuario/proyecto.git</code>
<code>git push origin <rama></code>	Envía los commits locales al repositorio remoto.	<code>git push origin main</code>
<code>git pull origin <rama></code>	Descarga y combina los últimos cambios del remoto con tu rama local.	<code>git pull origin main</code>
<code>git reset --hard <commit></code>	Revierte el repositorio al estado de un commit específico (⚠️ Destruye cambios posteriores).	<code>git reset --hard alb2c3d</code>
<code>git rm <archivo></code>	Elimina un archivo del repositorio y del sistema.	<code>git rm archivo.txt</code>
<code>git config --global user.name "Tu Nombre"</code>	Configura tu nombre de usuario global para Git.	—
<code>git config --global user.email "tuemail@example.com"</code>	Configura tu correo electrónico global.	—

📌 Consejos útiles:

- Siempre usa `git status` antes de hacer `commit` para verificar qué archivos cambiarán.
- Utiliza mensajes de commit claros: "Corrige error de autenticación", "Agrega validación al formulario", etc.

Conceptos Básicos de Tipos de Datos y Operadores en C#

Conceptos Básicos de Tipos de Datos y Operadores en C#

1. Introducción

El lenguaje C# (C Sharp) es un lenguaje de programación moderno, orientado a objetos y fuertemente tipado. Todas las variables deben tener un tipo de dato definido que indique qué valores puede almacenar y qué operaciones se pueden realizar.

Tipos de Datos en C#

En C#, los tipos de datos se dividen en **dos categorías principales**:

◆ A. Tipos de Valor (Value Types)

Guardan directamente el valor en memoria.

Tipo	Descripción	Tamaño	Ejemplo
byte	Entero positivo	8 bits	byte edad = 20;
int	Entero con signo	32 bits	int cantidad = 150;
long	Entero largo	64 bits	long poblacion = 8000000;
float	Decimal (precisión simple)	32 bits	float pi = 3.14f;
double	Decimal (precisión doble)	64 bits	double radio = 2.718281;
decimal	Decimal de alta precisión	128 bits	decimal precio = 99.99m;
char	Carácter Unicode	16 bits	char letra = 'A';
bool	Valor lógico	1 bit	bool activo = true;

◆ B. Tipos de Referencia (Reference Types)

Guardan una **referencia a un objeto** en memoria (no el valor directamente).

Tipo	Descripción	Ejemplo
string	Cadena de texto	<code>string nombre = "Marche";</code>
object	Tipo base de todos los tipos	<code>object dato = 42;</code>
array	Conjunto de valores del mismo tipo	<code>int[] numeros = {1,2,3};</code>
class	Define una clase (objeto complejo)	<code>class Persona { public string Nombre; }</code>

◆ C. Tipos Anulables (Nullable Types)

Permiten que los tipos de valor almacenen `null`:

```
int? edad = null;
```

⚙ 3. Operadores en C#

Los **operadores** permiten realizar operaciones matemáticas, comparaciones y decisiones lógicas.

📊 3.1 Operadores Matemáticos

Operador	Descripción	Ejemplo	Resultado
+	Suma	<code>5 + 2</code>	7
-	Resta	<code>5 - 2</code>	3
*	Multiplicación	<code>5 * 2</code>	10
/	División	<code>5 / 2</code>	2
%	Módulo (resto)	<code>5 % 2</code>	1

🔍 3.2 Operadores de Comparación

Operador	Significado	Ejemplo	Resultado
<code>==</code>	Igual a	<code>5 == 5</code>	true

Operador	Significado	Ejemplo	Resultado
!=	Distinto de	5 != 3	true
>	Mayor que	8 > 3	true
<	Menor que	3 < 8	true
>=	Mayor o igual	5 >= 5	true
<=	Menor o igual	4 <= 5	true

⚡ 3.3 Operadores Lógicos

Operador	Descripción	Ejemplo	Resultado
&&	AND (Y lógico)	true && false	false
	OR (O lógico)	true false	true
!	NOT (Negación)	!true	false

🔢 3.5 Operadores de Incremento y Decremento

Operador	Descripción	Ejemplo	Resultado
++i	Incremento antes de usar el valor	int i = 1; Console.WriteLine(++i);	Imprime 2
i++	Incremento después de usar el valor	int i = 1; Console.WriteLine(i++);	Imprime 1
--i	Decremento antes de usar el valor	int i = 2; Console.WriteLine(--i);	Imprime 1
i--	Decremento después de usar el valor	int i = 2; Console.WriteLine(i--);	Imprime 2
i+=n	Incrementa el valor en n cantidad funciona igual para los operadores de -, *, /	int i = 2; Console.WriteLine(i*=3);	Imprime 6

Tablas de Verdad

AND (&):

P	Q	P & Q
T	T	T
T	F	F
F	T	F
F	F	F

OR (||):

P	Q	P Q
T	T	T
T	F	T
F	T	T
F	F	F

NOT (!):

P	!P
T	F
F	T

Clase string

Namespace: using System;

Método / Propiedad	Descripción	Ejemplo	Resultado / Acción
Length	Devuelve la cantidad de caracteres.	"Hola".Length	4
ToUpper()	Convierte a mayúsculas.	"hola".ToUpper()	"HOLA"
ToLower()	Convierte a minúsculas.	"HOLA".ToLower()	"hola"
Substring(i, n)	Extrae una parte de la cadena.	"Hola".Substring(1,2)	"ol"
Contains("x")	Verifica si contiene una subcadena.	"programar".Contains("gram")	true
IndexOf("x")	Retorna la posición del primer carácter encontrado.	"Hola".IndexOf("l")	2
Replace("a", "o")	Reemplaza caracteres o palabras.	"casa".Replace("a", "o")	"coso"
Trim()	Elimina espacios en blanco al inicio y fin.	" hola ".Trim()	"hola"
Split(' ')	Separa la cadena en partes (array).	"uno dos tres".Split(' ')	["uno", "dos", "tres"]
Join(",", array) (estático)	Une elementos de un array en un string.	String.Join(",", arr)	"uno,dos,tres"

Método / Propiedad	Descripción	Ejemplo	Resultado / Acción
Equals ("x")	Compara dos cadenas.	"Hola".Equals ("hola")	false
StartsWith ("H") / EndsWith ("a")	Verifica inicio o fin.	"Hola".EndsWith ("a")	true
Reverse () (no directo)	Se usa con LINQ para invertir.	new string ("hola".Reverse () .ToArray ())	"aloh"

Clase Math

Namespace: using System;

Método / Propiedad	Descripción	Ejemplo	Resultado / Acción
Math.Abs (x)	Valor absoluto.	Math.Abs (-10)	10
Math.Pow (a,b)	Potencia.	Math.Pow (2,3)	8
Math.Sqrt (x)	Raíz cuadrada.	Math.Sqrt (9)	3
Math.Round (x)	Redondea al entero más cercano.	Math.Round (3.6)	4
Math.Floor (x)	Redondea hacia abajo.	Math.Floor (3.9)	3
Math.Ceiling (x)	Redondea hacia arriba.	Math.Ceiling (3.1)	4
Math.Max (a,b)	Devuelve el mayor.	Math.Max (3,5)	5
Math.Min (a,b)	Devuelve el menor.	Math.Min (3,5)	3
Math.PI	Constante de pi.	Math.PI	3.14159...
Math.Sin (x) / Math.Cos (x) / Math.Tan (x)	Funciones trigonométricas (en radianes).	Math.Sin (Math.PI/2)	1

Clase `DateTime`

Namespace: `using System;`

Método / Propiedad	Descripción	Ejemplo	Resultado / Acción
<code>DateTime.Now</code>	Fecha y hora actual.	<code>DateTime.Now</code>	27/10/2025 15:43:00
<code>DateTime.Today</code>	Solo la fecha actual.	<code>DateTime.Today</code>	27/10/2025 00:00:00
<code>AddDays (n) / AddMonths (n) / AddYears (n)</code>	Suma o resta tiempo.	<code>DateTime.Now.AddDays (5)</code>	Fecha + 5 días
<code>ToShortDateString ()</code>	Muestra solo la fecha.	<code>DateTime.Now.ToShortDateString ()</code>	"27/10/2025"
<code>ToLongTimeString ()</code>	Muestra solo la hora.	<code>DateTime.Now.ToLongTimeString ()</code>	"15:43:00"
<code>Year, Month, Day</code>	Obtiene componentes.	<code>DateTime.Now.Year</code>	2025
<code>Compare (a, b)</code>	Compara dos fechas.	<code>DateTime.Compare (f1, f2)</code>	-1, 0 o 1
<code>Parse ("2025-10-27")</code>	Convierte texto a fecha.	<code>DateTime.Parse ("2025-10-27")</code>	Objeto <code>DateTime</code>

Clase `Random`

Namespace: `using System;`

Método / Propiedad	Descripción	Ejemplo	Resultado / Acción
<code>new Random ()</code>	Crea un generador de números aleatorios.	<code>Random rnd = new Random ();</code>	—
<code>Next ()</code>	Devuelve un número entero aleatorio.	<code>rnd.Next ()</code>	Un entero aleatorio

Método / Propiedad	Descripción	Ejemplo	Resultado / Acción
Next(max)	Devuelve un número entre 0 y max-1.	rnd.Next(10)	0 a 9
Next(min, max)	Devuelve un número entre min y max-1.	rnd.Next(1, 6)	1 a 5
NextDouble()	Devuelve un número decimal entre 0 y 1.	rnd.NextDouble()	0.0 a 0.999...

Clase Array

Namespace: using System;

Método / Propiedad	Descripción	Ejemplo	Resultado / Acción
Sort(array)	Ordena los elementos.	Array.Sort(nums)	Ordena de menor a mayor
Reverse(array)	Invierte el orden.	Array.Reverse(nums)	Invierte los elementos
IndexOf(array, value)	Devuelve la posición del elemento.	Array.IndexOf(nums, 5)	Índice del 5
Length	Devuelve el tamaño.	nums.Length	Cantidad de elementos
Clear(array, start, count)	Reemplaza con valores por defecto.	Array.Clear(nums, 0, nums.Length)	Pone ceros

Clase List<T>

 *Namespace:* using System.Collections.Generic;

Método / Propiedad	Descripción	Ejemplo	Resultado / Acción
Add(item)	Agrega un elemento.	lista.Add(10)	Añade al final

Método / Propiedad	Descripción	Ejemplo	Resultado / Acción
Remove (item)	Elimina la primera coincidencia.	lista.Remove (10)	Quita 10
Count	Cantidad de elementos.	lista.Count	5
Contains (x)	Verifica si existe un elemento.	lista.Contains (3)	true/false
Sort ()	Ordena la lista.	lista.Sort ()	Orden ascendente
Reverse ()	Invierte el orden.	lista.Reverse ()	—
Clear ()	Limpia la lista.	lista.Clear ()	Vacía todo

Clase Console

Namespace: using System;

Método / Propiedad	Descripción	Ejemplo	Resultado / Acción
WriteLine ("texto")	Muestra texto y salta línea.	Console.WriteLine ("Hola")	Imprime "Hola"
Write ("texto")	Muestra texto sin salto de línea.	Console.Write ("Hola")	—
ReadLine ()	Lee texto ingresado por el usuario.	string nombre = Console.ReadLine ();	Guarda la entrada
ReadKey ()	Espera una tecla.	Console.ReadKey ();	Pausa el programa
Clear ()	Limpia la consola.	Console.Clear ();	Borra pantalla